DRONE SOIL-SAMPLING ROUTE OPTIMIZATION

Travelling Salesperson Problem using Nearest Neighbor and Branch and Bound

Java implementation | SDG 2: Zero Hunger | SDG 9: Industry & Innovation

Project Presentation

Problem Overview

Agricultural drone route planning

- A drone must visit every soil-sampling point and return to its starting base.
- Each movement has a travel cost or distance.
- The goal is to find a minimum-cost closed route.
- The problem is modeled as a Travelling Salesperson Problem (TSP).

Project Objectives

What the Java application is designed to do

- Accept point coordinates or an n × n distance matrix.
- Calculate Euclidean distances from coordinates.
- Use Nearest Neighbor to produce a fast baseline route.
- Use Branch and Bound with lower-bound pruning for exact small-instance search.
- Verify results against brute force, including an eight-location case.
- Report route, total cost, explored/pruned nodes, and draw a route view.

System Workflow

From input to route visualization

1 Choose input

Coordinates or distance matrix

2 Build costs

Euclidean distances or entered values

3 Baseline

Nearest Neighbor tour

4 Optimize

Branch and Bound + pruning

5 Verify

Brute force for small n

6 Display

Tour, cost, statistics, Swing view

Algorithms Used

A heuristic provides an initial solution; exact search improves it

Nearest Neighbor

- Start at the base.
- Repeatedly visit the cheapest unvisited point.
- Return to the base after visiting all points.
- Fast baseline; not guaranteed to be optimal.

Branch and Bound

- Build candidate tours one location at a time.
- Maintain the best complete route as an upper bound.
- Estimate a lower bound for each partial route.
- Prune a branch when it cannot beat the current best.

Brute-Force Verification

Independent correctness check for small instances

- Fix the starting location at the base.
- Enumerate every ordering of the remaining locations.
- Calculate the cost of each closed tour.
- Compare the minimum brute-force cost with Branch and Bound.
- For n = 8, the fixed-start search checks (8 − 1)! = 5,040 tours.

Symmetric vs. Asymmetric Costs

Both input types are supported

Symmetric matrix

d(i,j) = d(j,i)

Example: Euclidean distance between GPS points.

Asymmetric matrix

d(i,j) may differ from d(j,i)

Example: wind, one-way corridors, or direction-dependent energy use.

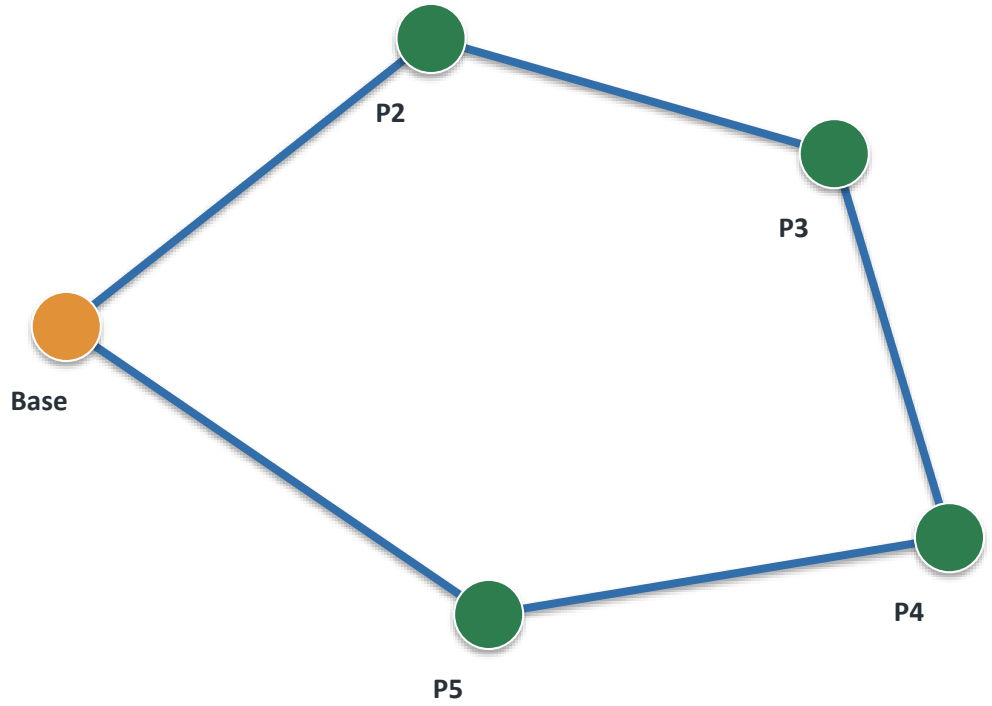
Complexity and NP-Hardness

Why exact optimization becomes difficult at scale

- TSP is NP-hard: no polynomial-time exact algorithm is known for all instances.
- Nearest Neighbor runs in $O(n^2)$ in this implementation.
- Brute force examines $(n - 1)!$ tours for a fixed starting point.
- Branch and Bound can prune many routes, but its worst-case search remains factorial/exponential.
- For large missions, use heuristics, local search, approximation methods, or specialized solvers.

Route Visualization

Java Swing graphical display



Illustrative route layout (not geographic scale)

The Java Swing window marks the base in red, sample points in green, and the selected tour in blue.

Testing Plan

Recommended test cases before submission

- Four-point square using coordinates: check a simple symmetric case.
- Eight-point symmetric matrix: compare Branch and Bound with brute force.
- Small asymmetric matrix: confirm direction-specific costs are used.
- Larger instance: record runtime and pruning statistics.
- Check that the displayed tour starts and ends at the base and visits each point once.

Limitations and Future Work

Moving from a classroom model toward a field-ready planner

- Current route drawing is schematic; future work can plot real GPS coordinates.
- Add no-fly zones, obstacles, wind, battery limits, and payload constraints.
- Improve lower bounds to prune more branches.
- For larger datasets, explore 2-opt/3-opt, genetic algorithms, or approximation methods.
- Support CSV import and route export for field operations.

CONCLUSION

The project applies TSP optimization to agricultural drone planning.

Nearest Neighbor provides a quick baseline. Branch and Bound searches for an optimal small-instance route, while brute force checks correctness. Supporting symmetric and asymmetric costs makes the model more flexible.

SDG 2 – Zero Hunger | SDG 9 – Industry, Innovation and Infrastructure

Thank You